



Session 3:

Build Your First PoC (Part-2)

03 **FINISH, DOCUMENT, STABILISE**

Documentation-First Development

Prompt Claude Code to write the README as you build — not after. A PoC without a README is not handoff-ready.



Forces scope clarity

Writing what it does exposes ambiguity. If you cannot describe it clearly, it is not done.



Makes handoff instant

Anyone picking this up — engineering, a colleague, future you — can start without a briefing.



Confirms the build

Generating the README reveals gaps between what you intended and what Claude actually built.

THE README PROMPT

```
"Write a README.md for this project. Include:  
- What it does (1 paragraph)  
- Who uses it and when  
- How to install and run it  
- What a successful run looks like  
- What is out of scope"
```

Run this now — before you finish building. Edit what is wrong. Keep it current.

Error Handling Basics

The difference between a demo that breaks under questions and one that holds up.

Invalid or missing input

Without: Script crashes with an unreadable stack trace **With:** Clear message: what was expected, what was received

"Add input validation. If [INPUT] is missing or invalid, print a clear error and exit gracefully."

API or service unavailable

Without: Connection timeout with no explanation to the user **With:** Informative failure: what was tried, that it failed, what to check next

"Handle API connection errors. If the request fails, name the endpoint and suggest checking the API key."

Empty or unexpected response

Without: KeyError or AttributeError mid-execution **With:** Handled: logs the empty response, continues or exits cleanly — no crash

"Handle empty responses from [API]. If no results, print a message and exit — do not crash."

Your PoC is done when it answers the question it was built to answer.

✗ Not when it handles every edge case

✗ Not when it is polished or production-ready

✗ Not when you have simply run out of time

Go back to your success criterion. If it passes — you are done.

The Self-Review Prompt

Before you call it done — ask Claude to stress-test it. 30 seconds. Saves your demo.

THE PROMPT

```
"What are the top 3 things most likely to break or fail in a live demo of this PoC? Be specific."
```

Classify each risk Claude identifies:

Blocking

Breaks the demo entirely

→ Fix it now — before the commit

Deferrable

Edge case unlikely in a controlled demo

→ Note it in the README. Move on.

Known Limitation

Out of scope by design

→ Confirm it is in your 'out of scope' list.

Saving Your Work — Git Commit

Commit before Session 3. Claude generates the message. You review and push.

Generate the commit message

1

```
$ claude "Write a Git commit message for this session. Include what it does, what it does not do, and any known limitations."
```

Review it. Edit if vague.

Stage your files

2

```
$ git add .
```

Check git status first. Only stage files you intended to change.

Commit with the generated message

3

```
$ git commit -m "[CLAUDE-GENERATED MESSAGE]"
```

This PoC runs through Sessions 3 and 4. A clear message matters.

Live Build Part 2 — Complete, Document, Commit

HANDS-ON LAB

Finish the PoC. Make it explainable. Make it committable.

1

Complete the working core

If Part 1 left anything unfinished — finish it. One function, one clean output.

2

Run the self-review prompt

Fix anything blocking. Note what is deferrable in the README.

```
$ claude "Top 3 things most likely to fail in a live demo?"
```

3

Generate and review your README

Run the README prompt. Fix any inaccuracies before saving the file.

```
$ claude "Write a README.md for this project..."
```

4

Add basic error handling

Apply the three error handling prompts to your most likely failure points.

5

Commit to Git

Generate commit message · Stage files · Commit · You are done for Session 2.

```
$ git add . && git commit -m "[message]"
```

✓ **SUCCESS: PoC runs · Self-review done · README saved · Error handling added · Committed to Git**

60-Second Demo

SHARE-OUT

One or two volunteers. 60 seconds each. Show it running — not on slides.

COVER THREE THINGS IN 60 SECONDS

1

What question does your PoC answer?

One sentence. The problem it was built to solve.

2

Show it running.

Execute it live. Real output. Not a screenshot or a description.

3

What would you change with another 30 minutes?

The most important gap you are aware of.

Everyone else: note one thing you would borrow from what you just saw.

What You Now Have

SCOPED & BRIEFED

- ✓ PoC brief: one journey, one integration, one testable outcome
- ✓ Scope test passed
- ✓ CLAUDE.md created and verified

BUILT & RUNNING

- ✓ Working PoC — core logic runs
- ✓ Build loop applied: prompt → run → fix → repeat
- ✓ Self-review prompt run — blocking risks addressed

DOCUMENTED & COMMITTED

- ✓ README generated and reviewed
- ✓ Basic error handling added
- ✓ Committed to Git with a descriptive message

This PoC continues in Sessions 3 and 4. Keep it open — you will build on it.

What's Next — Session 3

Debug, Review & Improve Your Code · Same PoC. New layer.

AI-Assisted Debugging

01

Apply the 5-step debug framework. Read stack traces with Claude as co-pilot. Find and fix the failures before they reach a stakeholder.

AI-Assisted Code Review

02

Run a structured review prompt on your own PoC. Identify severity-ranked issues. Apply the fixes. Re-run the review to confirm.

Development Workflows

03

Commit clean work to Git with Claude-generated messages. Produce a one-page technical overview your engineering team can act on.

Bring: your committed PoC · your Git branch · your prompt log from today.

Q&A



What is the correct **ORDER** for adding a feature to an existing telecom billing service?

- A. /plan → implement → /init → /diff
- ✓ B. /init + /memory → /plan → implement → /diff → /code-review
- C. implement → /code-review → /init → /plan
- D. /clear → implement → /security-review → /init

ANSWER

B Load context first, plan, implement, then inspect and review. Understanding precedes editing.

You must estimate effort on an unfamiliar EHR-integration repo in two days. Which approach gives the best-grounded estimate?

- A** A. Ask Claude to guess story points from the README alone
- B** B. Run `/code-review --fix` on everything and count the fixes
- ✓ C** C. `/init`, ask architecture questions, map modules and complexity, list unknowns, then size by component
- D** D. Immediately `/plan` the new feature and use the plan length as the estimate

ANSWER

C Understanding before sizing. Estimate by component with explicit unknowns; the PM owns the final number.

Before relying on an external tool (database, Jira) inside a session, which steps are appropriate? Select all that apply.

- ☒ **A. /mcp to set up or verify the server connection**
- ☒ **B. /permissions to set the approval rules you want**
- ☐ **C. /usage to check token spend**
- ☒ **D. Confirm the connector resolves to the right project or board**

ANSWER

A, B, D /mcp registers and verifies servers, /permissions sets trust boundaries, and you confirm it points to the right data. /usage is about spend, not tool setup.

The key difference when using Claude on front-end versus back-end work is:

- ☐ A. Claude can only do back-end
- ☒ B. Back-end is verified by running tests; front-end usually needs you to check the rendered UI, because Claude cannot see pixels
- ☐ C. Front-end needs no verification at all
- ☐ D. There is no practical difference

ANS

Two different verification loops. Claude reasons over code, not over pixels, so you close the loop on anything visual in a real browser.

A non-technical analyst wants to build a small tool with Claude. Best approach?

- ☐ A. Tell them to learn to code first
- ☒ B. Start from a plain-English brief and the build loop; Claude writes and explains, they verify the behaviour and the outcome
- ☐ C. Conclude they cannot use Claude
- ☐ D. Have them paste unknown code from the internet and hope

Automation · Augmentation

Claude augments the non-coder. They own the problem and the verification; Claude owns the syntax. Outcome over implementation.

Project estimation. You are a Program Manager with two days to estimate enhancements to a hospital DischargeFlow service. You have read access to the repo and Claude Code. Mark each correct step.

RIGHT

Run /init to generate CLAUDE.md and get a map of the stack, modules, and data stores.

RIGHT

Ask Claude architecture questions (@src) about request flow, dependencies, and where discharge logic lives.

RIGHT

Run /code-review --fix across the whole repo and base the estimate on the number of auto-fixes.

RIGHT

Have Claude flag unknowns and external integrations (HL7, FHIR) that widen the estimate

RIGHT

Size the work by component with optimistic, likely, and pessimistic ranges, and list assumptions.

RIGHT

Skip understanding and ask Claude to 'just give a number in story points' from the README

RIGHT

Treat Claude's number as final and commit the date to the client without human review.

Project estimation. You are a Program Manager with two days to estimate enhancements to a hospital DischargeFlow service. You have read access to the repo and Claude Code. Mark each correct step.

RIGHT

Run /init to generate CLAUDE.md and get a map of the stack, modules, and data stores. *A grounded map is the right starting point.*

RIGHT

Ask Claude architecture questions (@src) about request flow, dependencies, and where discharge logic lives. *Understanding drives a defensible estimate.*

WRONG

Run /code-review --fix across the whole repo and base the estimate on the number of auto-fixes. *Review is not sizing, and --fix mutates code you only have read access to assess.*

RIGHT

Have Claude flag unknowns and external integrations (HL7, FHIR) that widen the estimate. *Surfacing unknowns is exactly what a two-day estimate needs.*

RIGHT

Size the work by component with optimistic, likely, and pessimistic ranges, and list assumptions. *Ranged, assumption-based sizing is sound estimation practice.*

WRONG

Skip understanding and ask Claude to 'just give a number in story points' from the README. *An ungrounded number is a guess, not an estimate.*

WRONG

Treat Claude's number as final and commit the date to the client without human review. *The PM owns the estimate. Always verify before committing to a client.*

LESSON Understand and bound the work first; the PM owns the final number.

Testing a new feature. You are a QA Lead. Engineering shipped a 'partial refund' feature in the ReturnFlow checkout service. You have Claude Code. Mark each step.

- RIGHT** Read CLAUDE.md (or /init) to understand the service, then @ the new feature files to see what changed.
- RIGHT** Ask Claude to derive test cases from the acceptance criteria, including edge cases (zero amount, over-refund, currency rounding).
- RIGHT** Ask Claude to write code that breaks the live payment gateway to 'prove' it is fragile. .
- RIGHT** Have Claude generate unit and integration tests, then you run them and review the failures.
- RIGHT** Use /security-review on the new payment path.
- RIGHT** Accept 'all tests pass' from Claude without running the suite yourself.
- RIGHT** Ask Claude to map the feature's regression surface on existing flows before sign-off.

Testing a new feature. You are a QA Lead. Engineering shipped a 'partial refund' feature in the ReturnFlow checkout service. You have Claude Code. Mark each step.

RIGHT

Read CLAUDE.md (or /init) to understand the service, then @ the new feature files to see what changed. *Know the change surface before writing tests.*

RIGHT

Ask Claude to derive test cases from the acceptance criteria, including edge cases (zero amount, over-refund, currency rounding). *Edge-case coverage is the point of the exercise.*

WRONG

Ask Claude to write code that breaks the live payment gateway to 'prove' it is fragile. *That is attacking a production system. Claude will not help, and it is not testing.*

RIGHT

Have Claude generate unit and integration tests, then you run them and review the failures. *Generate fast, but you run and judge the results.*

RIGHT

Use /security-review on the new payment path. *Payment paths warrant a read-only security pass.*

WRONG

Accept 'all tests pass' from Claude without running the suite yourself. *Never sign off on a claimed result you did not execute.*

RIGHT

Ask Claude to map the feature's regression surface on existing flows before sign-off. *Regression impact is core QA diligence.*

LESSON Generate tests fast, but you run them and own the sign-off.

Thank You.



A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.



www.fractal.ai